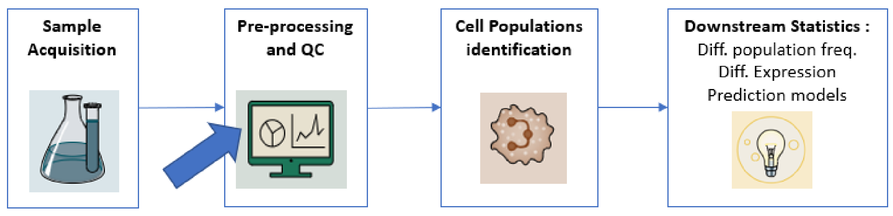
CytoPipeline: Building and Visualizing Pre-processing and Quality Control Pipelines for Flow Cytometry Data

Philippe Hauchamps[1], Dan Lin[2] and Laurent Gatto[1]

[1] Computational Biology and Bioinformatics, de Duve Institute, UCLouvain, Brussels, Belgium
[2] Department of Vaccines Research & Translational Statistics, GSK, Rixensart, Belgium

Pre-processing and Quality Control:



Pre-processing => several steps:

- ✔ Removal of margin events
- ✔ Compensation
- ✔ QC for signal stability
- ✔ Removal of doublets, debris, dead cells, etc.
- ✔ ...

CytoPipeline and CytoPipelineGUI packages:

⇒ facilitating the **set-up**, **run**, and **visualization of results** at each step:

- **explicit** and **centralized definition** of pipeline pre-processing steps (Figure 1)
- **user-defined** steps enabled through the `CytoPipeline` and `CytoProcessingStep` **S4 classes** (Figure 2)
- **caching of results of all steps** allowing direct access (*BiocFileCache* implementation)
- **interactive visualizations** with two **shiny apps** (Figures 3 & 4)

Challenges for the bioinformatician:

- combinatorial problem: choice of packages/methods for each step, method parameters, order of steps,...
- non standardized diagnostic and visualization tools
- lack of standardized and easy to use pipeline design tools

Figure 2: multiple `CytoProcessingStep` contained in a `CytoPipeline` object

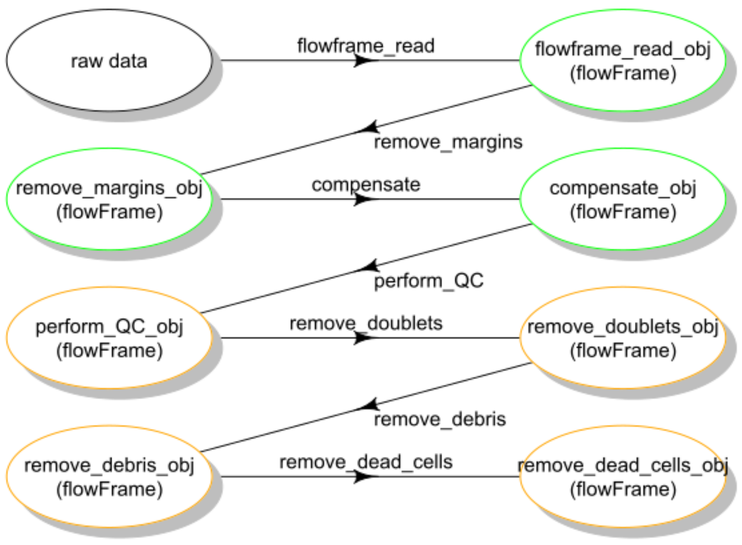


Figure 3: interactive visualization of scale transformations:

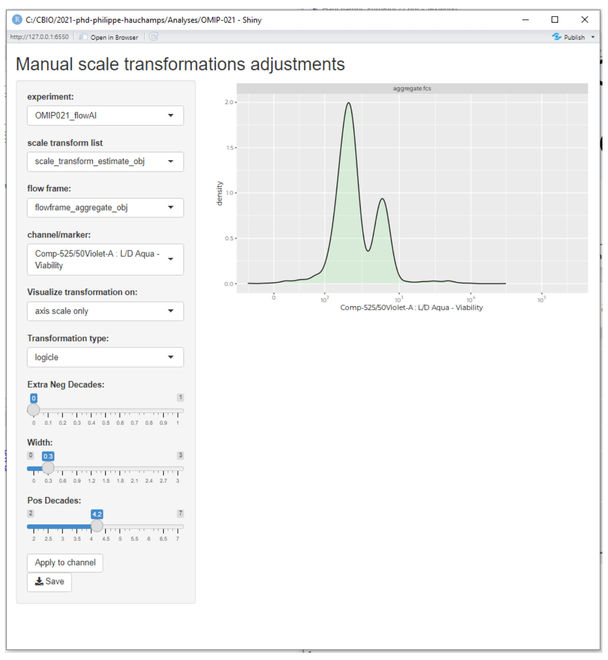


Figure 1: (left) definition of the pipeline steps in JSON and (right) R code loading and executing the pipeline

```
{
  "experimentName": ["OMIP021_PeacoQC"],
  "sampleFiles": ["./data/sample_Donor1.fcs",
                  "./data/sample_Donor2.fcs"],
  "flowFramesPreProcessingSteps": [
    {
      "name": ["flowframe_read"],
      "FUN": ["readSampleFiles"],
      "ARGS": {
        "truncate_max_range": [false],
        "min.limit": null
      }
    },
    {
      "name": ["remove_margins"],
      "FUN": ["removeMarginsPeacoQC"],
      "ARGS": []
    },
    {
      "name": ["compensate"],
      "FUN": ["compensateFromMatrix"],
      "ARGS": {
        "matrixSource": ["fcs"]
      }
    },
    {
      "name": ["perform_QC"],
      "FUN": ["qualityControlPeacoQC"],
      "ARGS": {
        "preTransform": [true],
        "min_cells": [150],
        "max_bins": [500],
        "step": [500],
        "MAD": [6],
        "IT_limit": [0.55],
        "force_IT": [150],
        "peak_removal": [0.3333],
        "min_nr_bins_peakdetection": [10]
      }
    },
    {
      "name": ["remove_doublets"],
      "FUN": ["removeDoubletsFlowStats"],
```

```
library(CytoPipeline)

# creation on CytoPipeline object,
# using json file as input
pipL_PeacoQC <-
  CytoPipeline("./myPipeline.json")

# execute pipeline
execute(pipL_PeacoQC, rmCache = TRUE)

# plot work flow graph
plotCytoPipelineProcessingQueue(
  pipL_PeacoQC,
  whichQueue = "pre-processing",
  sampleFile = 1)

# example of retrieving a flow frame
# at a given step
ff <- getCytoPipelineFlowFrame(
  pipL_PeacoQC,
  whichQueue = "pre-processing",
  sampleFile = 1,
  objectName = "remove_doublets_obj")
```

Figure 4: interactive comparison of `flowFrame` at different steps, within or between pre-processing pipelines:

This work was funded by GlaxoSmithKline Biologicals S.A., under a cooperative research and development agreement between GlaxoSmithKline Biologicals S.A. and the De Duve Institute (UC Louvain).
We would like to thank the European Bioconductor Society for providing Philippe Hauchamps with a travel scholarship to attend the EuroBioC2022 conference.